

Three-Stage Transformer Vibration Diagnosis

Code-Level Design Guide (V1 + V2)

Focus: dual-branch encoder (Hetero image + Zerone 1200-D features),
and three-stage unsupervised / semi-supervised workflow.

1. Big Picture: What the Three-Stage System Does

Goal: Starting from raw 8192-point transformer vibration waveforms, build a three-stage diagnosis pipeline that

- (1) learns a compact representation of "mostly normal" behaviour in a purely unsupervised way,
- (2) assigns anomaly scores and pseudo-labels to all samples, and
- (3) optionally trains a small supervised classifier head on a tiny labeled validation set, while keeping the test set strictly unlabeled during training.

Two code versions:

- V1 = single-branch Hetero pipeline (CWT+STFT+Context image, ResNet18 encoder, DeepSVDD+VAE).
- V2 = dual-branch pipeline: Hetero image branch + Zerone 1200-D feature branch, fused into a 512-D representation (HybridAnomalyModelV2).

The core objects to understand when reading the code are:

- ThreeStageConfig / ThreeStageConfigV2 – global hyper-parameters and paths
- DualBranchDataset – how JSON/JSONL vibration files become model inputs
- HybridAnomalyModel / HybridAnomalyModelV2 – encoder + DeepSVDD head + VAE head
- Stage-1 / Stage-2 / Stage-3 functions – orchestration of the three training phases.

2. Data Flow: DualBranchDataset and Feature Extraction (V2)

2.1 Input files and basic parsing

Both V1 and V2 assume JSON or JSONL files with records like:

```
{
  "data_time": "...", # timestamp or similar
  "signal_value": "0.12,0.10,...", # or a list of floats
  "sensor_id": "..."
}
```

Each file collects many timestamps, and each timestamp may have multiple sensors (multiple channels).

DualBranchDataset(root_dir, cfg, use_labels=False/True) scans a directory recursively, collects all *.jsonl and *.json, and builds an index of samples where

- one sample = all channels at a single time-stamp within a single file.

Key steps in _build_index():

- 1) For each file:
 - Read all records, group them by a time key (data_time / dataTime / timestamp / time).
- 2) For each time group:
 - Parse 'signal_value' into float32 arrays of length 8192 (pad/truncate if needed).
 - Collect them into a list [sig_ch0, sig_ch1, ..., sig_chU-1].
- 3) Store tuples (file_path, time_key, list_of_signals, label_or_None) into self.samples.

If use_labels=True, the label is inferred from path keywords such as 'normal', 'fault', '■■■', '■■■'; otherwise, label is set to -1.

2.2 From multi-channel signals to dual-branch inputs

At __getitem__ time, DualBranchDataset does three things for each time group:

(1) Channel aggregation:

- Stack all channels into shape (T, U).
- Compute per-channel energy $E_u = \text{mean}(\text{sig}_u^2)$.
- Normalise E_u into weights $w(u)$.
- Weighted sum: $x = X @ w$ (shape (T,)), a single 1-D "virtual channel".

(2) Hetero image branch:

- Apply z-score normalisation on x.
- Compute:
 - Ch0: CWT time-frequency map (Morlet wavelet).
 - Ch1: STFT magnitude spectrogram.
 - Ch2: "Context" image built by folding the waveform.
- Resize each channel to (H, W) = (224, 224), min-max norm each.
- Stack into a (3, 224, 224) float32 tensor.

(3) Zerone feature branch:

- Use zerone_features.py to compute

* Time-domain features : 15-D

3. Model Core: Dual-Branch Encoder and HybridAnomalyModelV2

3.1 Dual-branch encoder

The encoder in V2 has two branches:

- Image branch (CNN):
 - ResNet18 backbone (conv + bn + relu + maxpool + layer1..4).
 - AdaptiveAvgPool2d(1) + Flatten, producing a 512-D feature.
- Zerone branch (MLP):
 - Linear(1200 -> 512) + BN + ReLU
 - Linear(512 -> 256) + BN + ReLU

=> A 256-D feature.
- Fusion:
 - Concatenate 512-D (image) and 256-D (zerone) -> 768-D.
 - Linear(768 -> 512) + BN + ReLU.

=> Unified 512-D representation z_enc used by both SVDD and VAE.

3.2 SVDD head

Deep SVDD head stores a learnable (or initialised then fixed) center c in the 512-D latent space and defines anomaly scores as

$$\text{score_svdd}(x) = \| z_enc(x) - c \|^2.$$

In the training loop, the SVDD loss is essentially $\text{mean}(\text{score_svdd})$ plus optional soft-boundary terms.

3.3 Spatial VAE head

For interpretability and regularisation, V2 also contains a VAE-like decoder:

- μ_conv , $\logvar_conv$: 1x1 convolutions on top of a spatial feature map (for V1 this was the CNN branch only).
- Reparameterisation $z = \mu + \epsilon * \sigma$.
- Decoder: series of ConvTranspose2d layers, upsampling back to 3x224x224.

The VAE loss for a batch is

```
recon_loss = L1(recon, input_image) averaged over pixels;  
kl_loss = KL(N(mu, sigma) || N(0, I));  
loss_vae = recon_loss + beta * kl_loss.
```

3.4 Combined anomaly score

HybridAnomalyModelV2.anomaly_score(image, zerone) combines the two heads:

```
score_svdd from the Deep SVDD head, and  
score_vae = recon_loss + gamma * kl_loss from the VAE head.
```

After normalisation, the final anomaly score is

$$\text{score_final} = \alpha * \text{score_svdd} + (1 - \alpha) * \text{score_vae}.$$

4. Stage 1 (train_stage1_v2): Unsupervised Training

4.1 Data usage

Stage 1 treats both TRAIN and VAL directories as unlabeled:

```
train_ds = DualBranchDataset(TRAIN_DIR, cfg, use_labels=False)
val_ds = DualBranchDataset(VAL_DIR, cfg, use_labels=False)
combined = ConcatDataset([train_ds, val_ds])
```

This matches the idea that stage 1 should not look at any human labels, even if VAL actually contains labeled examples for later reference.

4.2 Pretraining the VAE

Before SVDD training, the code runs a short VAE pretraining phase:

- Optimiser only sees the VAE parameters and encoder parameters.
- For each batch:
img, zerone, _, _ = batch
out = model(img, zerone)
loss = recon_loss + 0.01 * kl_loss

This warms up the encoder + decoder so that reconstruction is already meaningful, preventing trivial SVDD collapse.

4.3 Initialising the SVDD center

After pretraining, the center c is initialised by averaging z_{enc} over all samples:

```
center = mean( z_enc(x) over all batches )
```

Tiny values near zero are nudged away from exactly 0 to avoid degenerate behaviour.

4.4 Joint SVDD + VAE training

Then the full HybridAnomalyModelV2 is optimised:

For each epoch $e = 1..E$:

```
beta = min(BETA_VAE, BETA_VAE * (e / BETA_WARMUP))
```

For each batch:

```
out = model(img, zerone)
svdd_loss = mean(score_svdd)
vae_loss = mean(recon_loss) + beta * mean(kl_loss)
total_loss = svdd_loss + vae_loss
```

Backpropagate total_loss, clip gradients if needed, step the optimiser, and update the cosine-annealing LR scheduler.

Track epoch-wise svdd_loss, vae_loss, total_loss in a history dict, and save stage1_best.pth when avg total_loss improves.

4.5 Outputs

5. Stage 2 (run_stage2_v2): Pseudo-Labels from Anomaly Scores

5.1 Data and score computation

Stage 2 reuses TRAIN + VAL (still with use_labels=False) and the frozen HybridAnomalyModelV2 to compute anomaly scores for every sample:

```
loader = DataLoader(ConcatDataset([train_ds, val_ds]), batch_size=..., shuffle=False)
```

For each batch:

```
img, zerone, _, idx = batch
score = model.anomaly_score(img, zerone)
append score and the sample indices.
```

The result is two arrays:

all_scores : anomaly score per sample
all_indices: global indices into the combined dataset

5.2 Percentile-based thresholds

Given all_scores, the code chooses two percentile thresholds

```
t_normal = percentile(all_scores, NORMAL_PERCENTILE) # e.g. 5%
t_anomaly = percentile(all_scores, ANOMALY_PERCENTILE) # e.g. 99%
```

and defines three sets:

```
pseudo_normal = { idx | score <= t_normal }
pseudo_anomaly = { idx | score >= t_anomaly }
uncertain = the others.
```

These are not yet used directly in V2's Stage 3, but conceptually they form a "pseudo-labeled pool" which could be combined with a small labeled VAL set for semi-supervised classification.

5.3 Outputs

Stage 2 saves a '.npz' or similar file containing:

- all_scores, all_indices
- pseudo_normal, pseudo_anomaly, uncertain
- t_normal, t_anomaly

It also generates a histogram of scores with vertical lines marking t_normal and t_anomaly, plus a pie chart of pseudo label proportions.

6. Stage 3: How V1/V2 Use Labels and What to Watch Out For

6.1 Original Stage 3 in V1/V2

train_stage3 / train_stage3_v2 currently do the following:

- 1) Build a labeled dataset from TEST_DIR with use_labels=True (labels inferred from folder names like 'normal', 'fault', etc.).
- 2) Split this TEST dataset into an 80% "train subset" and 20% "validation subset".
- 3) Construct a FaultClassifier (or FaultClassifierV2), which reuses the encoder part of HybridAnomalyModel(V2) and adds a small classification head on top of the 512-D representation.
- 4) Train this classifier on the 80% subset with cross-entropy loss, monitor val_acc / val_f1 on the 20% subset, and finally evaluate on the full TEST set.

This means that the TEST set is actually being used as labeled training data, not just as a blind evaluation set. From a strict ML perspective, this is "label leakage" if you intended TEST to be purely for final evaluation.

6.2 Recommended conceptual split (based on your latest idea)

To follow the rule

- TRAIN and TEST are used in an unsupervised way (no labels read),
- VAL is the only small labeled set used for threshold tuning or supervised heads,

one natural Stage 3 interpretation is:

Version A: Purely unsupervised evaluation

- Use the SVDD+VAE anomaly score and thresholds (t_normal, t_anomaly) from Stage 2.
- On VAL (with labels) only: sweep thresholds, choose a good operating point.
- Fix the thresholds.
- On TEST (without labels): apply the thresholds to produce predictions.
- Optionally, in a separate offline script, compare predictions with TEST labels without feeding them back into training.

Version B: Semi-supervised classifier (optional, more advanced)

- Use VAL labels as the only "trusted" labels.
- Optionally add high-confidence pseudo-labeled samples (from Stage 2) as extra training data with lower weight.
- Train a classifier head on top of the encoder, validating only on a held-out part of VAL.
- Use TEST strictly for forward-only evaluation later.

When reading or modifying Stage 3 code, always keep this separation in mind: where labels are read, and whether they influence parameters or just evaluation.

7. Practical Checklist While Reading / Modifying the Code

When you open `transformer_three_stage.py` (V1) or `transformer_three_stage_v2.py` (V2), you can use the following checklist to orient yourself quickly:

[1] Config

- Locate `ThreeStageConfig` / `ThreeStageConfigV2`.
- Check `ROOT` paths, `TRAIN/VAL/TEST` directories, `BATCH_SIZE`, `LR`, `epochs`, and `anomaly-percentile` settings.
- Confirm `DEVICE`, `BETA_VAE`, `alpha` for score fusion, etc.

[2] Dataset

- Find `DualBranchDataset` (V2) or the simpler `Dataset` in V1.
- Verify how one sample is defined (per timestamp, per file, per sensor set).
- Confirm how multi-channel aggregation is done and how 8192-point segments are padded/truncated.
- Check that `use_labels` is `False` in Stage 1 and Stage 2.

[3] Encoder + Heads

- In V1: image-only encoder feeding `DeepSVDD` + `SpatialResNetVAE`.
- In V2: dual-branch encoder feeding `DeepSVDD` + `VAE` with fusion.
- Understand `anomaly_score`: which components it mixes and how.

[4] Stage 1

- Ensure `VAE` pretraining, center initialisation, and combined loss are as expected.
- Note how histories are logged and where `stage1_best.pth` is saved.

[5] Stage 2

- Confirm `DataLoader` uses `TRAIN+VAL` with `use_labels=False`.
- Understand how thresholds `t_normal` and `t_anomaly` are selected (pure percentile vs. potential `MAD`-based methods).
- Locate where `pseudo_labels.npz` (or similar) is written.

[6] Stage 3

- Identify where labels are read (`use_labels=True`).
- Decide whether Stage 3 should be "pure evaluation" or "semi-supervised classifier" for your current experiment.
- If you want pure evaluation: avoid training a new classifier on `TEST` and only use `VAL` labels for threshold selection.

Once you align the code with this conceptual picture, both V1 and V2 become two concrete implementations of the same high-level three-stage diagnosis scheme.